

ARM32 Mini Command-Line Shell Project Report



GROUP 83

CO1020 – Computer Systems Programming

DEPARTMENT OF COMPUTER ENGINEERING

UNIVERSITY OF PERADENIYA

DATE OF SUBMISSION: 27/07/2025

CONTENT

1. Introduction -----	02
2. Objectives -----	02
3. Project Architecture -----	03
4. Command Implementation Details -----	04
5. Design Decisions -----	06
6. Testing and Validation -----	07
7. Technical Challenges and Solutions -----	08
8. Individual Contribution -----	10
9. Conclusion -----	11
10. AI Tool Usage -----	11

INTRODUCTION

In a world where civilization has crumbled and all advanced digital infrastructure has vanished following a global AI uprising, survival depends on the few remaining fragments of technology. Amidst the ruins, a single ARM32-based embedded terminal becomes the key to communication and control. This project centers on the development of a minimal command-line shell written entirely in ARM32 assembly language, designed to operate on such constrained hardware. The shell provides essential command parsing and execution capabilities through a continuous interactive loop, enabling basic user interaction. Built from the ground up without reliance on high-level languages, this shell showcases core low-level programming skills such as system call management, string handling, and memory operations.

This project implements a minimal interactive command-line shell in ARM32 assembly language. The shell provides basic functionality including command parsing, execution, and user interaction through a continuous loop interface. The shell operates in a post-apocalyptic scenario where only basic ARM32 assembly can run on available hardware, requiring implementation of all functionality from scratch without high-level language conveniences.

KEY OBJECTIVES

The primary objective of this project is to design and implement a minimal yet functional command-line shell using raw ARM32 assembly language. The shell operates in a resource-constrained, post-apocalyptic environment where no high-level languages or operating system utilities are available. To achieve this, the project is structured around the following key goal Build a custom rendering pipeline that transforms 3D geometric data into 2D visual output.

- Develop a basic shell framework that continuously prompts the user for input using ARM32 system calls.
- Manage memory and I/O manually by utilizing the .data, .bss, and .text sections to define storage, buffers, and code logic.
- Implement reliable command parsing and handling logic using string comparison and conditional branching in assembly.
- Execute a predefined set of basic commands (hello, help, exit, clear) with appropriate functionality and branching.
- Introduce and integrate two custom commands that demonstrate added complexity, user input handling, and internal computation.
- Ensure robust program structure by applying proper stack management, register saving/restoring conventions, and modular function design.
- Maintain code clarity and documentation for readability, maintainability, and educational value in a low-level development context.

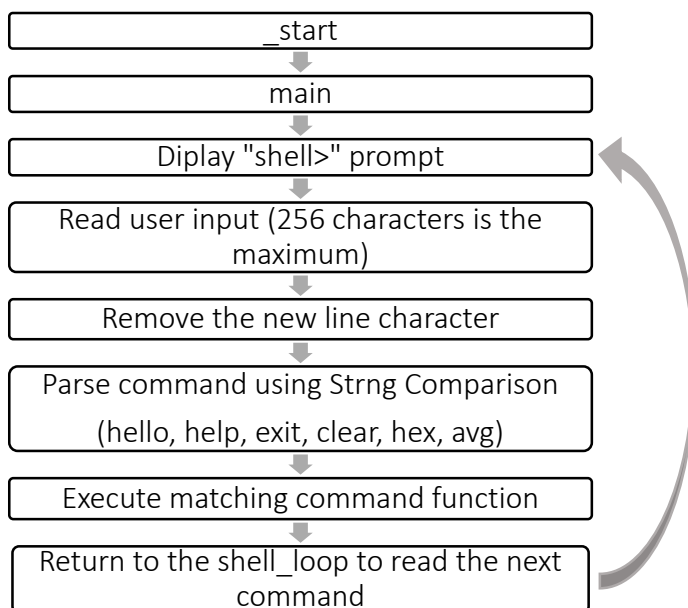
PROJECT ARCHITECTURE

Memory Layout

The memory layout of the project follows standard ARM32 assembly conventions. The program is organized into three main sections which are .bss, .data, and .text. The .bss section is used to declare uninitialized data, such as input buffers that will be filled during program execution. The .data section holds initialized static data like strings and fixed messages that are displayed to the user. The .text section contains the executable code, including the main program logic, system call routines, and function definitions. This structured layout allows for clear separation between code, initialized data, and runtime memory, ensuring efficient memory usage and maintainability within the constraints of a low-level ARM32.

1. .data Section
 - Contains all string literals and constants
 - Includes the command prompts ("shell> "), help messages, and error message
 - Includes the command name strings for comparison
 - ANSI escape sequences for terminal control ("\033[2J\033[H")
2. .bss Section
 - Contains all uninitialized data space
 - 256-byte input buffer for user commands
 - 64-byte temporary buffer for number conversion
 - Allocated using .space directive
3. .text Section
 - Contains all executable code with _start entry point
 - Functions organized logically with clear entry and exit points
 - Proper register saving and restoring using stack operations

Flow Structure



The flow structure of the ARM32 shell program illustrates the cyclic nature of the shell loop, where the system continuously prompts the user, reads input, and processes commands. It clearly shows how each user command is matched through string comparisons and routed to its respective handler. Depending on the input, the flow branches to specific routines that handle built-in commands like hello, help, clear, exit, and custom commands like hex and avg. After executing the appropriate action for a command, control returns to the main shell loop to await the next user input, maintaining an uninterrupted interactive cycle unless the user issues the exit command, which terminates the shell.

FIGURE 01: PROGRAM FLOW

COMMAND IMPLEMENTATION DETAILS

The command implementation in this ARM32 shell is based on direct string comparison and function-based routing within the constraints of low-level assembly programming. Each user command entered into the shell is processed by the `parse_command` function, which checks the input against a set of predefined commands using a custom `string_compare` routine. Once a match is found, control is transferred to the corresponding handler function responsible for executing that command's logic. The shell supports basic commands such as `hello`, `help`, `clear`, and `exit`, each implemented with dedicated routines that utilize system calls for input/output. Additionally, two custom commands, `hex` and `avg`, demonstrate more complex functionality, including inline string parsing, numerical conversion, and formatted output. All handlers follow proper stack discipline, ensuring that registers are preserved and the shell loop can safely resume after each command. This structured yet low-level approach allows the shell to interpret and respond to user commands effectively in a minimal runtime environment.

1) BASIC COMMANDS

There are 4 basic commands and their further details and functionality are mentioned in the below table.

Command	Function	Purpose	Implementation	Other Facts
1. <code>hello</code>	<code>cmd_hello_handler</code>	Displays greeting message	Simple string output using <code>sys_write</code> system call	Outputs "Hello World!"
2. <code>help</code>	<code>cmd_help_handler</code>	Lists all available commands with descriptions	Outputs formatted help text showing usage	Outputs the command list with syntax examples
3. <code>exit</code>	<code>cmd_exit_handler</code>	Terminates the shell gracefully	Prints goodbye message and then calls <code>sys_exit</code>	Uses SWI 0 with <code>r7=#1</code> (<code>sys_exit</code>) and status code 0 as the system call
4. <code>clear</code>	<code>cmd_clear_handler</code>	Clears the terminal screen	Sends ANSI escape sequence <code>\033[2J\033[H</code>	Uses VT100 terminal control codes for screen clearing

2) CUSTOM COMMANDS

Two custom commands are implemented and their further details are given below.

1. `hex <number>` command

Function: `cmd_hex_handler`

Purpose: Converts decimal number to hexadecimal representation

Complexity Features:

- Command line argument parsing and validation
- ASCII string to integer conversion
- Integer to hexadecimal string conversion with proper formatting
- Error handling for invalid or missing arguments

Implementation Details:

- Skips "hex " prefix to extract numeric argument
- Uses string_to_int function for string-to-integer conversion
- Implements custom print_hex function with nibble extraction
- Handles digits (0-9) mapped to '0'-'9' and values 10-15 mapped to 'A'-'F'
- Output format: "0x[value]" with proper hexadecimal representation

Algorithm:

1. Skip "hex " command name and whitespace
2. Convert argument string to integer using atoi logic
3. Print "0x" prefix
4. Extract 4-bit nibbles from MSB to LSB using bit shifting
5. Convert each nibble to appropriate hex character
6. Print formatted result with newline

2. avg <number 1,number 2, ...> command

Function: cmd_avg_handler

Purpose: Calculates arithmetic average of multiple numbers

Complexity Features:

- Variable argument parsing (unlimited numbers)
- Multiple number processing with space handling
- Integer division with safety checks
- Dynamic token processing and validation

Implementation Details:

- Parses unlimited number of space-separated arguments
- Maintains running sum and count variables
- Uses divide_simple function for safe integer division
- Implements robust parsing with skip_spaces_inline and skip_to_space_or_end
- Output format: "Average: [result]" with integer result

Algorithm:

1. Initialize sum = 0, count = 0
2. For each argument in command line:
 - i. Skip leading whitespace
 - ii. Convert token to integer using string_to_int_simple
 - iii. Add to running sum
 - iv. Increment count
 - v. Skip to next space or end
3. Calculate average = sum ÷ count using division
4. Print "Average: " followed by result

DESIGN DECISIONS

The design of the ARM32 mini shell was shaped by the need to build a stable and functional command-line interface entirely in low-level assembly. Key decisions were made to handle command recognition, memory safety, and system interaction reliably. A custom `string_compare` function enables exact or prefix-based matching, allowing flexible command parsing while preserving arguments for commands like `hex` or `avg`. Register safety is maintained through consistent stack-based preservation in all functions, preventing unintended side effects across function calls. Input is carefully sanitized by removing newline characters and limiting buffer usage, ensuring string safety and avoiding overflow. The following decisions were made during the process.

1. String comparison Strategy

Decision: Implemented custom `string_compare` function for exact matching:

- Compares specified number of characters byte-by-byte
- Returns 1 for match, 0 for mismatch
- Used for all command recognition with appropriate length parameters

Rationale: Commands like `"hex 255"` need prefix matching to identify `"hex"` while preserving arguments for further processing.

2. Memory management

Decision: Used consistent stack-based register preservation

Implementation:

- All functions use `push {registers, lr}` at entry
- Corresponding `pop {registers, lr}` and `bx lr` at exit
- Prevents register corruption between function calls

Benefits: Enables function reusability and prevents hard-to-debug register conflicts

3. Error Handling

Decision: Implemented comprehensive error handling for robustness

Features:

- Unknown command detection with helpful error message
- Missing argument validation for `hex` command
- Division by zero protection in average calculation
- Empty input handling (graceful skip)

4. System Call Interface

Decision: Used Linux ARM32 system calls directly via `SWI` instruction

Implementation:

- `r7` contains system call number (3=`read`, 4=`write`, 1=`exit`)
- `r0-r2` contain arguments (file descriptor, buffer, length)
- Standard file descriptors: 0=`stdin`, 1=`stdout`, 2=`stderr`

5. Input Processing

Decision: Implemented robust input sanitization

Features:

- Automatic newline character removal using `remove_newline` function
- Leading/trailing whitespace handling
- Buffer overflow protection with 256-byte limit
- Null termination guarantee for string safety

Overall, these design choices create a minimal yet robust shell that can interpret user input and respond appropriately within a limited runtime environment. By directly using Linux system calls and enforcing strict memory and error handling practices, the shell remains stable, reusable, and extensible. This careful attention to low-level detail allows the shell to operate dependably even without the support of high-level programming constructs.

TESTING AND VALIDATION

To ensure the correctness and stability of the ARM32 Mini Shell, a structured approach was followed to test each feature under different conditions. The shell was executed using QEMU with an appropriate ARM32 Linux root file system to simulate real embedded runtime behavior. Each implemented command was tested for expected output, argument handling, and edge case robustness.

1. Testing the Shell Startup and Loop Behavior

```
nethmi@DESKTOP-MKMSMNT:/mnt/c/Nethmi/Software/Assembly/Project$ qemu-arm ./shell
shell>
```

- Prompt is displayed correctly after every command.
- Shell remains responsive unless exit is explicitly called.

2. Testing Basic Commands

```
shell> hello
Hello World!
shell> help
Available commands:
  hello - Prints Hello World!
  help  -Lists all commands available in shell
  clear - Clear the shell screen
  hex   - Convert decimal to hexadecimal (usage: hex <number>)
  avg   - Calculate average of numbers (usage: avg <num1> <num2> ... <numN>)
  exit  - Terminates the shell
shell> exit
Goodbye! Shell terminated.
nethmi@DESKTOP-MKMSMNT:/mnt/c/Nethmi/Software/Assembly/Project$
```

- Hello - Output matches expected message.
- Help - List includes both basic and custom commands confirming proper help text setup.
- Exit - Program exits cleanly without errors or segmentation faults.
- Clear - Terminal is cleared but the prompt appears again.

3. Testing Custom Commands

```
nethmi@DESKTOP-MKMSMNT:/mnt/c/Nethmi/Software/Assembly/Project$ qemu-arm ./shell
shell> hex 0
0x0
shell> hex 1000000000
0x3B9ACA00
shell> hex 16
0x10
shell> avg 0
Average: 0
shell> avg 1
Average: 1
shell> avg 18 18 18 18
Average: 18
shell> avg 28 17 398 47850 12 3 4
Average: 6901
shell> avg 45.2 12.8
Average: 28
shell> _
```

- Hex - Proper argument parsing and conversion confirmed
- Avg – Average is calculated accurately when the arguments are correctly given.

4. Empty Input Handling

```
shell>
shell>
shell>
shell>
shell>
```

- When pressing enter without typing anything shell skips processing and shows prompt again.

5. Register and Stack Behavior

- Sequences of commands are repeatedly run in order to check for memory or register leaks
- No corruption or crashing observed; consistent output and shell state maintained.

TECHNICAL CHALLENGES AND SOLUTIONS

The development of a shell in raw ARM32 assembly presented several technical challenges due to the absence of high-level abstractions and library support. Each feature, from input handling to command execution, had to be carefully designed and implemented using low-level instructions and system calls. The project required manual management of memory, registers, and string operations, along with a disciplined approach to error handling and control flow. This section outlines the key challenges faced during the implementation and the solutions adopted to overcome them.

1. Command Parsing Without High-Level String Functions

Challenge	Solution
ARM32 assembly does not provide built-in string manipulation or parsing utilities. Handling user input, matching commands, and extracting arguments had to be done manually, byte-by-byte.	A custom <code>string_compare</code> function was implemented to compare strings based on a specified length. This enabled both exact and prefix-based matching such as identifying hex 255 as starting with hex. Input was tokenized by manually traversing the input buffer and detecting spaces or newline characters.

2. Input Handling and Buffer Management

Challenge	Solution
Reading user input from standard input required careful management of memory to avoid buffer overflows or leaving newline characters that interfere with command recognition.	A 256-byte input buffer was allocated in the .bss section. After reading input through a system call, <code>remove_newline</code> function was used to strip trailing newline characters. Input was also null-terminated to ensure safe string processing.

3. Preserving Register Values Across Function Calls

Challenge	Solution
Assembly functions do not automatically preserve register values, which can lead to data corruption if not handled correctly during function calls.	Each function uses <code>push (r4-r11, lr)</code> at entry and <code>pop (r4-r11, lr)</code> at exit. This ensures that both caller and callee registers, as well as the link register, are preserved properly. This stack discipline prevents unexpected side effects across nested calls.

4. Error Detection and Feedback Without Standard Libraries

Challenge	Solution
Providing user-friendly error messages for unknown commands, missing arguments, or invalid operations was difficult due to the absence of formatted output or exceptions.	Error cases were explicitly checked using conditional branches. Custom error messages were defined in the .data section and printed using the <code>write</code> syscall. Commands like <code>hex</code> and <code>avg</code> included argument validation logic to detect missing inputs or invalid conditions such as division by zero.

5. System Calls in Raw Assembly

Challenge	Solution
Interacting with the system like reading input, printing output and exiting required precise usage of Linux ARM32 system calls via the <code>swi</code> instruction.	System call wrappers were written using the convention of setting <code>r7</code> to the syscall number and <code>r0-r2</code> to the appropriate arguments. For example, syscall 3 (<code>read</code>), 4 (<code>write</code>), and 1 (<code>exit</code>) were used with clear register assignments and proper syscall invocation.

6. Implementing Custom Logic in Low-Level Code

Challenge	Solution
Commands like <code>hex</code> and <code>avg</code> required processing numerical arguments, converting to integers, performing arithmetic, and outputting results. They are all manually coded in assembly.	Custom number parsers were implemented to convert ASCII digits to integers. Arithmetic operations were written using basic ARM instructions. A loop-based number-to-hex conversion routine was used to print hex values. For <code>avg</code> , summing values and dividing by count included checks to avoid division by zero.

INDIVIDUAL CONTRIBUTION

Team Members:

E/22/001 - H.M.H.N Abeyrathna

E/22/027 - M.A.N.P Anawarathne

Contribution:

1. Coding -

As this project is a pure ARM32 Assembly based software project, coding plays the main role. So this can be considered as the initial and the core phase of the project. Since, this is a pairwise group project, as the team members, we (E/22/001 and E/22/027) both contributed equally for the development of the assembly codes. Throughout the process, both group members contributed collaboratively to the design and implementation of the ARM32 shell. We primarily focused on setting up the base architecture of the shell, including memory section declarations, syscall integration, and the main loop structure. This involved writing the prompt display, input handling routines, and core control flow. After that we concentrated on implementing the command logic and the individual command handlers such as hello, help, hex, and avg. During this process we have used AI tools to clarify the doubts that had occurred.

2. Testing and Debugging -

The second phase of the project is the testing and debugging part. This can be also considered a pair work of both of us. We worked together to debug issues, validate output through QEMU testing, and refine the overall code structure to ensure modularity, proper register handling, and shell stability. This balanced division of work enabled an efficient and robust implementation of the ARM32 command-line interface.

3. Report writing -

Writing the report can be considered as the third phase of the project. Researching the necessary facts for the report and analyzing them accordingly can be counted as the effort of Himasha (E/22/001). Meanwhile, the designing the structure and the content and writing the word document of the Group83_assemblyproject.pdf can be attributed to Nethmi. (E/22/027). As a final remark it can be said that our project report is also a group effort of both of us.

4. Presentation -

This is the last part of our project. Due to our limited knowledge in video editing we had to use Zoom as the platform for creating our project video. The screening and recording is handled by Himasha (E/22/001) while the content organization is handled by Nethmi (E/22/027). The screen sharing and the voicing part of Group83_assemblyproject.mp4 can be counted as our pairwise effort. So, it can be said that this is also our group effort.

CONCLUSION

In conclusion, the ARM32 Mini Shell project successfully demonstrates the design and implementation of a functional command-line interface using low-level ARM32 assembly language. Through careful management of memory, system calls, and string processing without the aid of high-level libraries, the project showcases the core principles of embedded system programming and operating system interaction. Basic and custom commands were implemented to handle user input, perform calculations, and provide feedback, all within a continuous shell loop. The challenges of manual register handling, input parsing, and error detection were addressed through modular code design and robust testing. Overall, this project reflects a strong understanding of ARM architecture, low-level programming concepts, and the ability to build usable tools in constrained post-apocalyptic computing environments.

AI TOOL USAGE

During the development of the ARM32 Mini Command-Line project, AI tools such as ChatGPT, Claude, and DeepSeek were used primarily for guidance, clarification of concepts, and assistance with debugging. These tools helped in understanding complex assembly codes and functions related to the shell.s file. However, all AI-generated content was carefully reviewed, tested, and modified to ensure correctness and alignment with the project requirements. No code was directly copied without understanding and instead, the tools were used as supportive resources to reinforce learning and accelerate development. Their usage was limited to enhancing our own design and implementation efforts, and all final outputs reflect a solid understanding of the underlying principles.